

Relazione Tecnica Approfondita: Architettura e Fasi di Esecuzione

progettoASM - Studio di Sistemi Agentici in Reti Sociali

Senior Software Engineer & Technical Writer

7 ottobre 2026

Indice

1	Panoramica del Progetto	2
2	Stack Tecnologico	2
3	Esecuzione della Pipeline: Analisi delle Fasi	2
3.1	Fase 0: Setup Ambiente, Data Ingestion e Baseline	3
3.1.1	Approfondimento: i moduli del sottosistema <code>graph</code>	3
3.1.2	Validazione Strutturale del Campionamento	6
3.1.3	Metriche Baseline del Sottografo	7
3.2	Fase 1: Livello Cognitivo e Agenti LLM	8
3.2.1	Approfondimento: i moduli del livello cognitivo	8
3.3	Fase 2: Dinamiche di Rete e Co-evoluzione	10
3.3.1	Approfondimento: <code>orchestrator.py</code> — Il Motore Centrale	11
3.3.2	Dinamiche Osservate in Fase 2	12
3.4	Fase 3: Intervento e Fact-Checking	13
3.4.1	Il Flusso di Esecuzione della Fase 3	14
3.4.2	Moduli Richiamati nella Fase 3	14
3.4.3	Risultati: Intervento a Confronto con il Controllo	16
3.4.4	Conclusioni sull'Efficacia dell'Intervento	16
4	Conclusioni e Prospettive	17
4.1	Visualizzazione della Rete: Topologia Iniziale vs Finale	17
4.2	Conclusioni Generali	18
4.3	Riepilogo Esecutivo della Pipeline	19

1 Panoramica del Progetto

Il repository [progettoASM](#) implementa un framework sperimentale avanzato per lo studio di un sistema agentico all'interno di reti sociali complesse. Concepito come progetto accademico per il corso di Analisi di Social Network e Media, il sistema si discosta dai tradizionali simulatori statici adottando un approccio di **co-evoluzione dinamica**.

Il problema centrale esplorato dal progetto è la modellazione realistica di come gli stati interni degli agenti (opinioni testuali e cognitive), la topologia della rete (legami sociali) e gli interventi esterni (come il fact-checking) si influenzino reciprocamente e simultaneamente. Invece di eseguire questi step come pipeline sequenziali e indipendenti, il framework li integra in un ciclo chiuso di interazione continua.

Il target di riferimento include ricercatori nell'ambito della *Network Science*, studiosi di sistemi multi-agente e data scientist. L'uso del dataset `ogbl-collab` (Open Graph Benchmark), originariamente una rete di co-autoraggio accademico, viene qui sfruttato in modo innovativo per rappresentare un ambiente sociale in cui la prossimità nello spazio vettoriale cognitivo degli agenti determina la probabilità di formazione o recisione dei legami.

2 Stack Tecnologico

Lo stack tecnologico è fortemente orientato al Machine Learning geometrico e al Natural Language Processing generativo, combinando astrazioni algoritmiche complesse con paradigmi di orchestrazione moderni.

- **Linguaggio e Core:** Python (≥ 3.10), scelto per l'ecosistema predominante in ambito Data Science.
- **Graph Neural Networks (GNN):** Utilizzo del modello *GraphSAGE* (Sample and Aggregate). La scelta di SAGE rispetto a GCN standard permette un apprendimento induttivo, fondamentale per grafi la cui topologia muta a ogni tick temporale, evitando di dover riaddestrare la rete da zero per i nuovi nodi o archi.
- **Agenti Generativi (LLM):** Motore di inferenza basato su **vLLM** (versione 0.30.0), che serve il modello `casperhansen/llama-3-8b-instruct-awq` distribuito su due GPU T4 in *tensor parallel*. vLLM massimizza il *throughput* grazie al continuous batching e ottimizza l'uso della VRAM, garantendo un'esecuzione rapida e massiva degli agenti senza incorrere nei *rate-limit* tipici delle API cloud esterne e mantenendo la completa sovranità sui dati. Con questa configurazione uno step di simulazione richiede circa 6 minuti.
- **Influenza e Propagazione:** Implementazione del modello *Linear Threshold* per la dinamica di contagio (diffusione di un'opinione/stato), accoppiato a una selezione greedy dei nodi *seed* per l'intervento di fact-checking. L'obiettivo di selezione è allineato alle stesse soglie che governano le transizioni nella simulazione; in alternativa è disponibile l'algoritmo CELF (*Cost-Effective Lazy Forward*) su modello Independent Cascade.

3 Esecuzione della Pipeline: Analisi delle Fasi

L'esecuzione del framework è strutturata in un *notebook* orchestratore (`kaggle_full_run.ipynb`) suddiviso logicamente in quattro fasi distinte, progettate per gestire l'intero ciclo di vita della co-evoluzione del sistema agentico.

Questo schema fornisce una chiave di lettura trasversale alle quattro fasi descritte di seguito: la Fase 0 prepara il grafo e le baseline su cui il ciclo opererà, la Fase 1 attiva il livello cognitivo (nodo "Agenti Generativi"), la Fase 2 chiude il ciclo tra livello cognitivo e livello strutturale (archi 1-4), e la Fase 3 introduce il livello di intervento (archi 5-6), completando il loop.

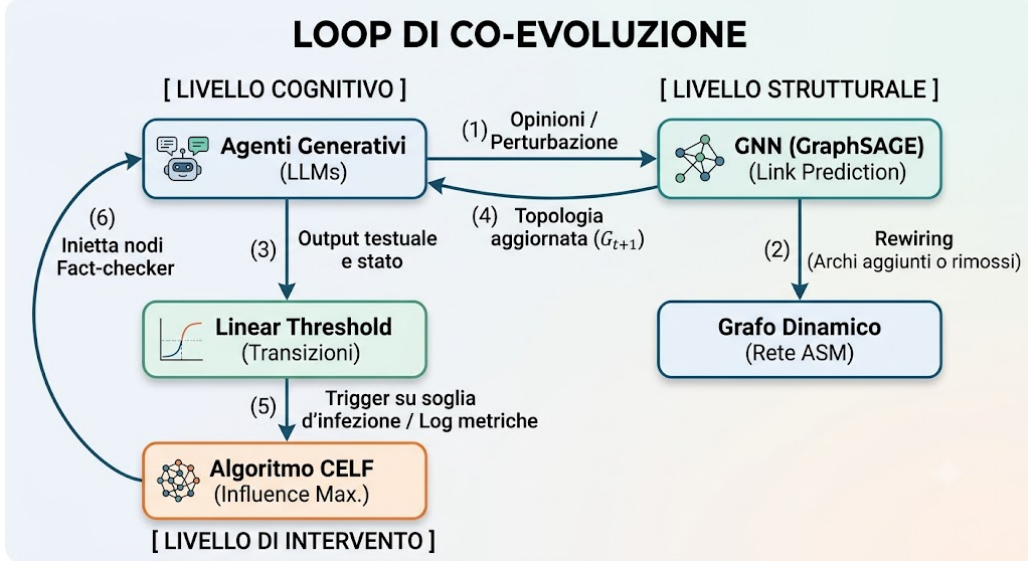


Figura 1: Schema d'insieme del loop di co-evoluzione. I tre livelli del sistema — cognitivo (Agenti LLM), strutturale (GNN e Grafo Dinamico) e di intervento (Linear Threshold e CELF) — interagiscono in un ciclo chiuso: gli agenti generano opinioni che perturbano gli embedding (1), la GNN calcola il rewiring della topologia (2), il nuovo output testuale alimenta la macchina a stati (3), la topologia aggiornata G_{t+1} retroagisce sugli agenti (4), le transizioni innescano il trigger su soglia d'infezione per CELF (5), che inietta i nodi Fact-Checker richiudendo il ciclo (6).

3.1 Fase 0: Setup Ambiente, Data Ingestion e Baseline

La Fase 0 si occupa della preparazione dell'infrastruttura computazionale e della profilazione iniziale della rete.

- **Setup Infrastrutturale:** Viene inizializzato in background il server vLLM locale caricando il modello `casperhansen/llama-3-8b-instruct-awq`. Le versioni delle librerie (`vllm`, `transformers`, `huggingface_hub`, `tokenizers`) sono fissate per garantire un ambiente riproducibile. I flag di avvio ottimizzano la memoria della GPU (es. `-gpu-memory-utilization 0.90`) e distribuiscono il modello su entrambe le GPU (`-tensor-parallel-size 2`), mentre l'accesso è protetto tramite variabili d'ambiente ricavate dai *Kaggle Secrets*. Se il server non risponde, il notebook interrompe l'esecuzione invece di proseguire senza modello linguistico.
- **Campionamento Topologico:** Il modulo `data_loader` importa il dataset completo `ogbl-collab`. Per renderne trattabile l'elaborazione, l'`extractor` preleva un sottografo target di 5000 nodi applicando un **Forest Fire Sampling** (con parametro `forward_prob = 0.5`). Questo algoritmo preserva in modo soddisfacente la distribuzione dei gradi, le *community* e il coefficiente di clustering originali.
- **Community Detection e Baseline:** Tramite l'algoritmo di *Label Propagation*, vengono identificate 415 community con un coefficiente di Modularity (Q) estremamente alto (0.8265). Il modulo `metrics` archivia i KPI di partenza, tra cui l'Echo Chamber Index e le metriche di centralità (Degree, PageRank, Katz).

3.1.1 Approfondimento: i moduli del sottosistema graph

La Fase 0 si appoggia a quattro moduli fondamentali del sottosistema **graph**, i quali forniscono rispettivamente le funzionalità di acquisizione del dataset, campionamento del sottografo, community detection e calcolo delle metriche strutturali e di opinione.

data_loader.py — Data Loader & Download Questo modulo gestisce l’acquisizione del dataset accademico standard `ogbl-collab` dall’Open Graph Benchmark (OGB) e lo adatta per la simulazione.

- **Caricamento e caching:**

- Cerca prima se esiste il file binario serializzato `full_graph.gpickle` in `data/processed/`. Se lo trova, lo carica all’istante, evitando di riscaricare e ricostruire il grafo e risparmiando minuti di elaborazione.
- Se non è presente, scarica il dataset in `data/raw/` tramite l’API `LinkPropPredDataset` di OGB.

- **Conversione e costruzione del grafo:**

- Converte la matrice di collegamenti di OGB (`edge_index`) in un grafo non orientato `NetworkX` (`nx.Graph`).
- Associa a ciascun nodo la feature vettoriale `Word2Vec` a 128 dimensioni, che rappresenta i temi di ricerca degli autori.

- **Filtro temporale:** se `dataset.year_filter` è impostato in `config.yaml`, elimina tutti gli archi creati dopo quell’anno, consentendo di studiare la rete in una specifica finestra storica.

- **Statistiche rapide** (`graph_summary`): restituisce informazioni descrittive istantanee sul grafo completo, tra cui numero di nodi/archi, densità, numero di componenti connesse e dimensione della componente connessa principale (LCC).

community.py — Community Detection Questo modulo si occupa di dividere la rete sociale in gruppi (o partizioni) di nodi densamente connessi tra loro, note come comunità. All’interno del framework, queste comunità costituiscono la struttura topologica di riferimento rispetto alla quale si misura la segregazione strutturale della rete.

- **Algoritmi di partizionamento supportati:**

- *Louvain* (`_louvain`): algoritmo euristico che ottimizza in modo iterativo la modularità (Q). Utilizza la funzione nativa di `NetworkX` ed è lo standard per individuare partizioni stabili.
- *Label Propagation* (`_label_propagation`): algoritmo basato sul consenso dei vicini, in cui ogni nodo assume l’etichetta maggioritaria dei suoi adiacenti. È più rapido di *Louvain* ma meno deterministico.

- **Meccanismo di caching:** per evitare il calcolo superfluo ad ogni avvio della simulazione, la funzione principale `detect_communities` verifica l’esistenza del file `community_map.json`. Se presente, valida che le dimensioni del grafo corrispondano e carica le comunità dalla cache; altrimenti avvia il calcolo e scrive il file di cache.

- **Analisi statistica** (`community_stats`): calcola e restituisce statistiche aggregate sulla qualità e sulla dimensione delle comunità trovate (numero totale di gruppi, dimensione minima, massima, media, deviazione standard e distribuzione dettagliata delle frequenze).

extractor.py — Subgraph Extractor Il dataset completo `ogbl-collab` contiene 235 868 nodi e 967 632 archi non orientati (2 358 104 collaborazioni con timestamp prima della deduplicazione). Simulare una rete agentica LLM su questa scala richiederebbe risorse insostenibili: il modulo `extractor.py` campiona un sottografo connesso, compatto e strutturalmente rappresentativo per l’esecuzione in locale o in cloud.

- **Algoritmi di campionamento:**

- *Forest Fire* (`_forest_fire_sample`, default): algoritmo probabilistico ispirato agli incendi boschivi. Preserva bene le proprietà della rete originale (legge di potenza dei gradi, coefficiente di clustering e struttura di comunità).
- *BFS* (`_bfs_sample`): esplorazione in ampiezza a partire da un nodo seme, tende a creare “sfere” dense.
- *Random Walk con Restart* (`_random_walk_sample`): cammini casuali con probabilità di riavvio (RWR) dal nodo seme iniziale, per evitare di rimanere bloccati in componenti dense.
- *Random Nodes* (`_random_nodes_sample`): campionamento casuale di nodi, che induce però sottografi disconnessi.

- **Funzioni principali:**

- `extract_subgraph`: isola la componente connessa gigante (LCC) del grafo originale per assicurare la massima connettività, estrae il sottografo indotto dai nodi campionati con la strategia scelta, rinumeri i nodi del sottografo (da 0 a $n-1$), allinea la matrice delle feature Word2Vec associate e salva i due artefatti (`subgraph.gpickle` e `embeddings.npy`).
- `_get_seed_node`: seleziona il nodo di partenza per BFS/Forest Fire, potendo scegliere l’hub principale della rete (nodo ad alto grado, `high_degree`) o un nodo casuale (`random`).

metrics.py — Graph Metrics È la “dashboard” di monitoraggio analitico del sistema. Calcola sia metriche di centralità dei nodi (utilizzate per scegliere i pazienti zero e i bersagli dell’intervento) sia indici strutturali e di opinione a ogni step temporale.

- **Metriche di centralità** (`compute_centralities`):

- *Degree Centrality*: quanti collegamenti diretti ha un autore.
- *PageRank*: determina l’importanza di un nodo basandosi sui link in ingresso provenienti a loro volta da nodi importanti; supporta l’accelerazione hardware su GPU tramite cuGraph, se disponibile.
- *Katz Centrality* (`_katz_centrality_sparse`): estensione del grado che considera tutti i cammini, penalizzandoli esponenzialmente in base alla lunghezza. È implementata da zero in modalità sparsa (power iteration) per evitare il blocco del processo dovuto al fill-in catastrofico delle matrici.
- *Betweenness Centrality*: quante volte un nodo si trova sul cammino minimo tra altre coppie di nodi (intermediari); viene calcolata su un campione di nodi per motivi di performance.

- **Metriche topologiche** (`compute_topological_metrics`): calcola densità, grado medio, numero di componenti connesse, coefficiente di clustering locale e diametro approssimato del grafo (`_approx_diameter`) mediante BFS campionate.

- **Metriche strutturali di segregazione:**

- *Modularità (Q)* (`compute_modularity`): misura quanto la rete resta divisa secondo le community individuate in Fase 0.
- *Echo Chamber Index (ECI)* (`compute_echo_chamber_index`): frazione media dei collegamenti di ciascun nodo che restano all’interno della sua community di Fase 0.

Entrambe sono metriche puramente strutturali: non dipendono dalle opinioni degli agenti e misurano la segregazione rispetto alla partizione iniziale della rete.

- **Metriche di opinione:** a ogni nodo viene associato un valore di opinione secondo la codifica $S = 0, I = +1, R = F = -1$, coerente con la direzione in cui gli stati spingono gli embedding.
 - *Opinion Homophily* (`opinion_homophily`): frazione di archi che collegano due nodi con la stessa opinione.
 - *Belief Assortativity* (`belief_assortativity`): coefficiente di assortatività sulle opinioni; valori positivi e crescenti indicano la formazione di echo chamber ideologiche, cioè gruppi connessi di agenti che la pensano allo stesso modo.
 - *Belief Polarisation* (BP) (`compute_belief_polarisation`): varianza delle opinioni normalizzata al massimo teorico, che misura se la rete è spaccata in posizioni contrapposte o se sta convergendo verso un consenso comune.
- **Punto di accesso** (`compute_all_metrics`): raggruppa tutte le metriche strutturali e di opinione per esportarle nel logger JSONL e nel CSV. Le metriche topologiche più costose vengono riutilizzate tra uno step e l'altro solo se l'insieme degli archi è rimasto identico; il controllo avviene tramite un'impronta dell'insieme degli archi (circa 13 ms su 26 000 archi), indispensabile perché con il rewiring a densità costante il numero di archi non cambia mai.

3.1.2 Validazione Strutturale del Campionamento

Per garantire che il sottografo a 5000 nodi estratto tramite Forest Fire Sampling sia effettivamente rappresentativo della rete originale, la Fase 0 include uno studio comparativo delle proprietà strutturali tra il grafo completo (limitato alla componente connessa principale, LCC) e il sottografo campionato.

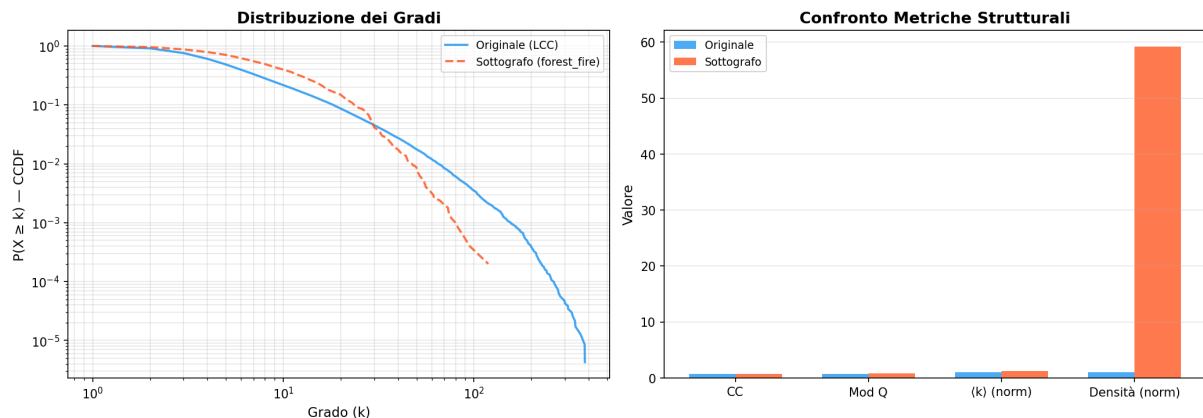


Figura 2: Confronto tra il grafo originale (LCC) e il sottografo campionato tramite Forest Fire Sampling. A sinistra, la distribuzione dei gradi in scala log-log (CCDF), che mostra come il sottografo preservi la forma a legge di potenza tipica delle reti scale-free. A destra, il confronto tra le principali metriche strutturali normalizzate (coefficiente di clustering, modularità, grado medio e densità).

Il grafico di sinistra riporta la *Complementary Cumulative Distribution Function* (CCDF) del grado dei nodi $P(X \geq k)$ in scala doppiamente logaritmica: l'andamento pressoché lineare di entrambe le curve conferma che sia il grafo originale sia il sottografo seguono una distribuzione dei gradi a coda pesante, coerente con la struttura scale-free attesa per una rete di co-autoraggio accademico. Il grafico di destra confronta invece, in forma normalizzata, il coefficiente di clustering (CC), la modularità (Mod Q), il grado medio $\langle k \rangle$ e la densità: quest'ultima risulta nettamente più alta nel sottografo, come atteso per effetto della riduzione del numero di nodi a parità di connettività locale preservata.

La tabella seguente riporta il confronto quantitativo dettagliato tra le due reti:

Metrica	Originale	Sottografo	Δ relativo
Nodi	232 865	5 000	−97.9%
Archi	961 883	26 246	−97.3%
Densità	0.000035	0.002100	+5819.6%
Grado medio $\langle k \rangle$	8.3	10.5	+27.1%
Grado max	382	119	−68.8%
$\sigma(\text{grado})$	13.4	9.7	−27.1%
Clustering CC	0.7204	0.6988	−3.0%
Modularity Q	0.6957	0.8265	+18.8%
N. comunità	23 115	415	−98.2%

Tabella 1: Validazione strutturale — confronto tra il grafo originale (LCC) e il sottografo estratto tramite Forest Fire Sampling.

Il rapporto di campionamento risulta pari al 2.1% dei nodi originali. Dal punto di vista computazionale, il calcolo delle metriche sul grafo completo ha richiesto circa 114 secondi, contro appena 0.4 secondi per il sottografo, con un guadagno di efficienza di oltre due ordini di grandezza. Nel complesso, i valori di clustering e grado medio si mantengono coerenti tra le due reti (scostamenti contenuti entro il $\pm 30\%$), a conferma della bontà del campionamento; l'aumento della modularità nel sottografo (+18.8%) è in parte un effetto atteso della riduzione di scala, che tende a rendere più nette le partizioni in comunità più piccole e numericamente meno frammentate.

3.1.3 Metriche Baseline del Sottografo

Una volta completata l'estrazione, il modulo `metrics` calcola l'intero set di metriche baseline sul sottografo di lavoro (5000 nodi), che costituiscono il punto di riferimento ($t=0$) rispetto al quale verrà misurata l'evoluzione della rete nelle fasi successive.

Metrica	Valore
Numero di nodi	5 000
Numero di archi	26 246
Densità	0.0021
Grado medio $\langle k \rangle$	10.4984
Componenti connesse	1
Clustering medio	0.6988
Grado massimo	119
Grado minimo	1
$\sigma(\text{grado})$	9.7465
Modularity Q	0.8265
Echo Chamber Index (ECI)	0.8318

Tabella 2: Metriche strutturali baseline del sottografo (Fase 0, $t = 0$).

Il grafo risulta composto da un'unica componente connessa, condizione necessaria affinché la diffusione dell'informazione e il rewiring strutturale possano interessare potenzialmente l'intera rete. Il valore di Echo Chamber Index già elevato in partenza (0.8318), in linea con l'alta modularità (0.8265), indica che il sottografo campionato eredita una segregazione comunitaria marcata già nella rete di co-autoraggio originale, ancor prima che il sistema agentico inizi a introdurre dinamiche di polarizzazione.

Il modulo calcola inoltre la classifica dei nodi per PageRank, utilizzata come uno dei criteri di selezione dei pazienti zero nella Fase 1 (Tabella 3).

Rank	ID Nodo	PageRank
1	3342	0.0011
2	1316	0.0009
3	505	0.0009
4	3612	0.0008
5	863	0.0008

Tabella 3: Top-5 nodi per PageRank sul sottografo baseline.

3.2 Fase 1: Livello Cognitivo e Agenti LLM

Nella Fase 1 si istanziano i processi logici che governano il singolo agente.

- **Inizializzazione Pazienti Zero:** Tramite il modulo `seeder`, il sistema seleziona 750 nodi (il 15% della rete) come propagatori iniziali. La selezione avviene tramite una logica *combined* per massimizzare la topologia di diffusione.
- **Orchestrazione Agenti:** A ogni step si attiva il 15% dei nodi; gli agenti attivi inviano richieste concorrenti (fino a 150 contemporanee, raggruppate in batch dal server vLLM) verso il modello LLM.
- **Generazione di Opinioni:** Gli agenti valutano le proprie metriche, le posizioni dei propri vicini e restituiscono l’opinione testuale che altera il proprio *feature vector*, azionando internamente la macchina a stati.

3.2.1 Approfondimento: i moduli del livello cognitivo

La Fase 1 si fonda su cinque moduli chiave, che insieme costituiscono il livello cognitivo del sistema: l’agente vero e proprio, il client LLM, il modulo di seeding dei pazienti zero, la macchina a stati e il gestore del grafo dinamico.

agent.py — Agente Cognitivo Rappresenta l’entità cognitiva elementare associata a ogni singolo nodo del grafo sociale. Ciascun agente incapsula la memoria, il posizionamento sociale (comunità, centralità) e la logica decisionale guidata da LLM.

- **Ciclo di vita del singolo step:**

1. *Percezione* (`prepare_step`): interroga il `NetworkManager` per estrarre la cronologia recente dei post pubblicati dai suoi contatti (feed) e calcolare la distribuzione degli stati del vicinato.
2. *Cognizione* (`cognize`): prepara i prompt di sistema e utente strutturati e invia la richiesta al modulo `LLMClient`, ottenendo un output JSON standardizzato.
3. *Azione e transizione* (`finalize_step`): esegue il parsing del JSON (estraendo opinione testuale, livello di suscettibilità e intenzione di condivisione); se lo desidera, l’agente pubblica un post, calcola la perturbazione vettoriale dell’embedding del nodo e aggiorna lo stato discreto consultando la macchina a stati.

- **Perturbazione degli embedding:** le transizioni di stato spostano l’embedding del nodo lungo una direzione “ideologica” comune a tutta la rete: gli Infetti in un verso, Resistenti e Fact-Checker nel verso opposto, con una piccola componente individuale (10%). In questo modo la similarità tra embedding, e quindi la link prediction della GNN, riflette la vicinanza di opinione tra agenti.

- **Ottimizzazioni per grandi grafi:**

- *Fast Path Optimization*: se l'agente è Suscettibile (S) e non ha alcun vicino infetto o fact-checker adiacente, la chiamata all'LLM e la costruzione del prompt vengono del tutto saltate, poiché non riceve alcuna pressione di diffusione e rimarrebbe stabile in S.
- *Smart Cache*: se lo stato e il feed del vicinato sono identici allo step precedente, riutilizza la risposta LLM in memoria senza inviare una nuova richiesta di rete.

llm_client.py — Wrapper LLM Portabile Gestisce la comunicazione diretta con i modelli di linguaggio (le API cloud di Gemini o i server di inferenza locali compatibili con OpenAI, quali Ollama e vLLM) in modo robusto e protetto.

- **Funzionalità di rete**: implementa retry automatici con backoff esponenziale per gestire disconnessioni temporanee o limiti di rate-limiting API, e pulisce/valida l'output testuale per garantire la corretta decodifica in formato JSON.
- **Riproducibilità**: ogni richiesta porta un seed derivato dal contenuto del prompt, così che due run che ricevono lo stesso prompt ottengano la stessa risposta. È la condizione che rende confrontabili la run con intervento e quella di controllo della Fase 3.
- **Caching su disco (LLMDiskCache)**: salva le risposte LLM su file JSONL persistente in modo thread-safe. Se la simulazione viene interrotta a metà (ad esempio superando il limite di tempo su Kaggle), al riavvio riutilizza le risposte già registrate per non perdere progresso.
- **Controllo dei costi (TokenBudget)**: un singleton globale che accumula ad ogni chiamata il consumo di token (sia in input che in output), dotato di una soglia di avviso (**warn_at**) e di una soglia di arresto forzato (**hard_limit**) per prevenire addebiti monetari indesiderati sulle API.

seeder.py — Seeding dei Pazienti Zero Si occupa di individuare e infettare i nodi iniziali ("Pazienti Zero", stato I) per innescare e far propagare la narrazione polarizzante all'interno del sottografo.

- **Strategie di selezione (select)**:
 - *pagerank/degree/katz/betweenness*: seleziona i top- k nodi aventi rispettivamente i punteggi più alti per queste metriche (influencer globali, ponti strutturali o hub locali).
 - *combined* (default): assegna a ogni nodo uno score pesato normalizzato (40% PageRank, 40% Katz, 20% Degree) per identificare i nodi con il massimo potenziale di impatto locale e globale.
 - *cross_community*: ripartisce i semi equamente tra tutte le comunità Louvain rilevate, selezionando il nodo con lo score combinato più alto all'interno di ciascuna, per coprire uniformemente il grafo.
 - *random*: selezione casuale riproducibile tramite seed.
- **Iniezione (inject)**: forza lo stato iniziale dei nodi selezionati a I (Infected) e forza la pubblicazione del primo post di propaganda per attivare il feed dei rispettivi vicini.

state_machine.py — Macchina a Stati Governa le transizioni di stato discrete degli agenti ($S \rightarrow$ Suscettibile, $I \rightarrow$ Infetto, $R \rightarrow$ Resistente, $F \rightarrow$ Fact-Checker) implementando un modello Linear Threshold (LT) esteso.

- **Soglie individuali**: ogni nodo u ha una soglia θ_u che dipende soltanto dal seed globale e dall'identificativo del nodo. La soglia è quindi identica in ogni sessione e dopo ogni ripresa da checkpoint, indipendentemente dall'ordine in cui i nodi vengono interrogati.

- **Modulazione cognitiva delle soglie:** nei modelli classici LT, un nodo si infetta se la frazione di vicini infetti supera una soglia fissa θ_u . Qui la soglia è riscalata in base all’output qualitativo dell’agente LLM:

$$\theta_{\text{eff}} = \theta_u \times (1.5 - \text{susceptibility})$$

Se l’agente dichiara alta suscettibilità, la sua soglia scende, rendendolo più vulnerabile all’influenza dei vicini.

- **Regole di transizione principali:**

- $S \rightarrow I$: avviene se la pressione del vicinato supera la soglia efficace ($\text{fraction_I} \geq \theta_{\text{eff}}$) e l’agente stesso propone il cambio di stato nel JSON.
- $S \rightarrow R$ (resistenza spontanea): avviene se l’agente, esposto all’infezione (almeno il 12% di vicini infetti), ha bassa suscettibilità e rifiuta la condivisione.
- $S \rightarrow R$ (inoculazione): avviene se la frazione di vicini Fact-Checker raggiunge `fc_protection_threshold` (0.10), non è inferiore alla frazione di vicini infetti e l’agente non propone di passare a I.
- $I \rightarrow R$: avviene se la frazione di vicini Fact-Checker raggiunge `fc_resistance_threshold` (0.25) oppure se l’agente corregge autonomamente la sua visione.
- $R \rightarrow I$ (Ricaduta): possibile ma molto rara, innescata solo con almeno il 60% di vicini infetti e una suscettibilità cognitiva molto alta (> 0.8).

network_manager.py — Gestore del Grafo Funge da unico layer di astrazione e persistenza per la rete sociale dinamica. Qualsiasi modulo esterno che desideri interrogare o alterare la struttura del grafo deve transitare per il `NetworkManager`.

- **Post Store:** memorizza la lista di post scritti da ciascun nodo e costruisce la vista del feed per gli adiacenti (`get_feed`), ordinandoli in modo decrescente per step temporale.
- **Topologia e rewiring:** aggiunge archi (`add_edge`), rimuove archi (`remove_edge`) e gestisce le transizioni strutturali in blocco (`apply_rewiring`) calcolate dalla GNN.
- **Gestione vettoriale:** custodisce la matrice degli embedding dei nodi, che vengono perturbati dagli agenti ad ogni iterazione fornendo il segnale dinamico su cui lavora la GNN; consente aggiornamenti massivi ottimizzati (`perturb_embeddings_batch`).
- **Codifica delle opinioni:** espone un’unica codifica numerica degli stati (`STATE_TO_BELIEF`: S = 0, I = +1, R = F = -1), usata da tutte le metriche di opinione.
- **Checkpointing:** salva (`save`) e ripristina (`load`) lo stato completo della simulazione (grafo `NetworkX`, post, stati ideologici e matrice degli embedding) su file pickle.

3.3 Fase 2: Dinamiche di Rete e Co-evoluzione

La Fase 2 innesca il ciclo ricorsivo vero e proprio, supervisionato dal `SimulationOrchestrator`.

- **Graph Rewiring (GNN):** La GNN elabora i feature vector alterati dagli agenti. Attraverso il modulo `rewirer`, la rete subisce variazioni dinamiche a densità costante: ogni nuovo arco tra agenti con score di link prediction elevato sostituisce l’arco esistente con lo score più basso. Il rewiring avviene con un intervallo di *cooldown* di 2 step e sostituisce al massimo 50 archi per volta.
- **Transizione e Contagio:** Contemporaneamente alla GNN, si verifica un aggiornamento degli stati via *Linear Threshold*. A ogni step il 15% dei nodi della rete valuta l’influenza cumulativa del vicinato e si adegua se la pressione sociale supera la propria resistenza individuale.

- **Checkpointing Resiliente:** L'Orchestratore compie frequenti backup di stato su file serializzati (.pkl) e in `metrics_history.csv`. Il sistema ripristina il *loop* dal checkpoint più recente per superare i vincoli temporali dell'ambiente cloud, agganciando i log pregressi alle nuove run; se il percorso indicato non esiste, il notebook cerca automaticamente checkpoint e CSV tra i dati in input della sessione Kaggle.

3.3.1 Approfondimento: `orchestrator.py` — Il Motore Centrale

Il modulo `orchestrator.py` definisce la classe `SimulationOrchestrator`, che rappresenta il motore centrale del framework. Ha il compito di coordinare tutte le fasi del loop co-evolutivo a ogni step temporale, gestire il checkpointing/resume e far comunicare i vari sottosistemi (grafi, agenti LLM e GNN).

Inizializzazione (`build_from_config`) Costruisce la simulazione partendo dall'oggetto `Config`, seguendo una sequenza precisa:

1. imposta i seed globali per la riproducibilità;
2. carica il sottografo campionato e le comunità;
3. inizializza gli embedding dei nodi (Word2Vec);
4. istanzia il `NetworkManager` come unica sorgente di verità;
5. calcola le centralità e usa il `Seeder` per infettare i pazienti zero iniziali;
6. ripristina lo stato precedente (Resume) se rileva checkpoint validi su disco, insieme a mappa delle comunità e metriche cumulative;
7. istanzia gli `Agent`, leggendo lo stato effettivo di ciascun nodo dal grafo ripristinato, e i relativi client LLM;
8. crea il modello `GraphSAGEModel`, il `GNNTrainer` e il `Rewirer`.

Il loop principale (`run` e `_run_step`) Il metodo `run` esegue un ciclo temporale per N passi, ripartendo dallo step successivo all'ultimo completato. A ogni singolo step `_run_step(t)`, l'orchestratore esegue in sequenza i tre cicli interni descritti di seguito, calcola le metriche aggregate, salva lo stato nel log JSONL e nel CSV, e scrive un checkpoint periodico.

I tre cicli interni dell'Orchestratore

- **Ciclo Agenti** (`_agent_cycle`): è il ciclo più critico a livello prestazionale a causa delle chiamate LLM (latenza di inferenza). Per risolverlo, l'Orchestratore implementa un design a tre fasi:
 1. *PREPARE* (sequenziale): legge in memoria dal `NetworkManager` e costruisce i messaggi di prompt per l'LLM di ogni nodo; fase puramente in memoria, richiede frazioni di secondo.
 2. *DISPATCH* (parallela): invia le richieste `llm.chat()` dei nodi in parallelo tramite un thread pool (`ThreadPoolExecutor`), nascondendo la latenza di I/O; i nodi inattivi vengono saltati grazie alla Fast Path Optimization e alla Smart Cache. Se fallisce più del 50% delle chiamate (`max_llm_failure_rate`), lo step viene interrotto prima di modificare lo stato della rete, e l'ultimo checkpoint resta valido.
 3. *FINALIZE* (sequenziale): riceve i risultati dell'LLM, esegue il parsing del JSON, valuta il cambio di stato tramite `StateMachine` ed esprime l'opinione nel post.

Come ottimizzazione, invece di aggiornare l'embedding nodo per nodo, il ciclo raccoglie tutti i delta di perturbazione e li applica alla fine in un'unica operazione vettorizzata tramite `perturb_embeddings_batch`.

- **Ciclo GNN** (`_gnn_cycle`): preleva la matrice degli embedding dei nodi aggiornata dagli agenti, esegue un passo di addestramento (`GNNTrainer.train_step`) sul grafo corrente per allineare la rete neurale alle nuove posizioni ideologiche dei nodi, ed estrae le predizioni di link prediction (`predict_links`) sulle coppie candidate. I candidati sono coppie a distanza 2 generate a partire da 500 nodi sorgente, al massimo 5 per sorgente; le sorgenti cambiano a ogni step, così che il rewiring esplori nel tempo l'intera rete.
- **Ciclo di Rewiring** (`_rewiring_cycle`): ad ogni intervallo di cooldown, interroga il `Rewirer` fornendogli i punteggi predetti dalla GNN e gli stati correnti degli agenti. In modalità `swap` (default) il `Rewirer` abbina ogni arco da aggiungere (nuovo legame omofilo) alla rimozione dell'arco esistente con lo score più basso, mantenendo costante la densità; un vincolo di sicurezza impedisce di isolare un nodo, tenendo conto anche delle rimozioni già decise nello stesso step. I cambiamenti vengono comunicati al `NetworkManager` per mutare la topologia fisica del grafo.

Moduli richiamati dall'Orchestratore L'Orchestratore unifica e richiama moduli provenienti da quattro diversi sottosistemi:

- **Sottosistema graph** (gestione rete e metriche): `network_manager.py` (`NetworkManager`), che custodisce la topologia fisica, i post e gli embedding, gestendo le transizioni topologiche del rewiring; `metrics.py` (`compute_centralities`, `compute_all_metrics`), che calcola l'importanza dei nodi all'avvio e, alla fine di ogni step, le metriche strutturali (modularità, Echo Chamber Index) e di opinione (omofilia, assortatività, polarizzazione).
- **Sottosistema agents** (logica cognitiva degli autori): `agent.py` (`Agent`, `AgentStepContext`), che incapsula il comportamento decisionale dell'agente; `seeder.py` (`Seeder`), che trova i nodi hub iniziali per l'iniezione dei semi ideologici; `state_machine.py` (`StateMachine`), che calcola la transizione Linear Threshold modulata dalla suscettibilità dell'utente; `llm_client.py` (`LLMClient`, `MockLLMClient`), interfaccia per le chiamate al modello o al mock usato nei test.
- **Sottosistema gnn** (rappresentazione e link prediction): `embeddings.py` (`EmbeddingManager`), che carica ed esporta la matrice di embedding Word2Vec; `model.py` (`GraphSAGEModel`), il modello GraphSAGE implementato in PyTorch Geometric (o NumPy per fallback); `trainer.py` (`GNNTrainer`), che gestisce il loop di ottimizzazione e predice i link candidati per il rewiring; `rewirer.py` (`Rewirer`), che seleziona gli archi da aggiungere e da rimuovere a partire dagli score della GNN.
- **Sottosistema utils** (log, checkpoint e determinismo): `logger.py` (`SimLogger`), che scrive lo storico delle metriche e transizioni in formato JSONL; `checkpoint.py` (`CheckpointManager`), che serializza e deserializza lo stato completo del framework ad intervalli regolari, ordinando e potando i checkpoint per numero di step; `seed.py` (`set_all_seeds`), che imposta i seed casuali per garantire la riproducibilità degli esperimenti.

3.3.2 Dinamiche Osservate in Fase 2

L'esecuzione documentata usa l'LLM reale in tutte le fasi. La Fase 2 comprende 100 step (0-99), eseguiti in due sessioni Kaggle da 50 step ciascuna con ripresa da checkpoint. Il file `metrics_history.csv` registra l'evoluzione dello stato della rete ad ogni step.

Il grafico di sinistra mostra un tipico andamento epidemico. La popolazione Suscettibile decresce da 3965 a 978 unità; gli Infetti superano i Suscettibili già allo step 16, crescono rapidamente nei primi 30 step e poi si stabilizzano intorno al 41% dei nodi (picco di 2034 infetti allo

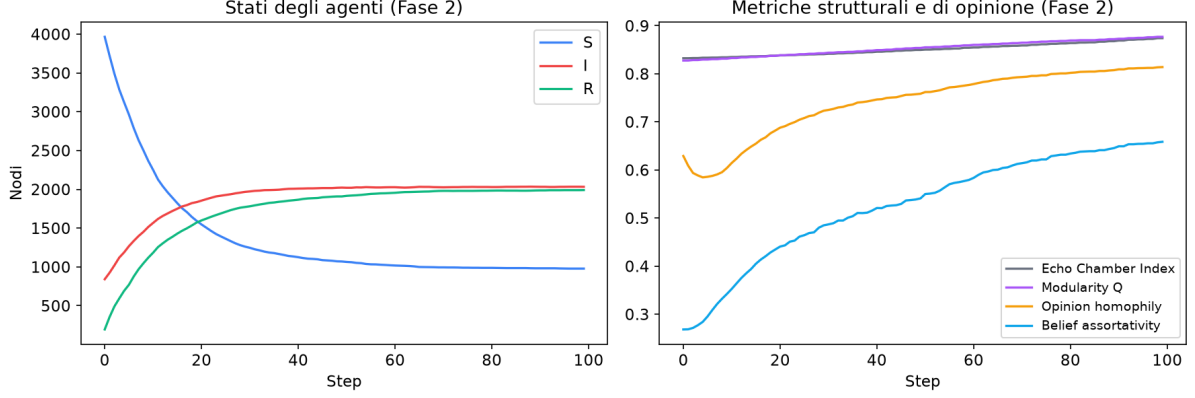


Figura 3: Evoluzione della rete durante la Fase 2 (step 0–99). A sinistra, la dinamica delle popolazioni di agenti negli stati S (Suscettibile), I (Infetto) e R (Resistente). A destra, l’evoluzione delle metriche strutturali (Echo Chamber Index, Modularity Q) e di opinione (Opinion homophily, Belief assortativity).

Step	S	I	R	ECI	Mod. Q	Op. homophily	Belief assort.
0	3 965	841	194	0.832	0.828	0.629	0.268
25	1 363	1 927	1 710	0.840	0.841	0.708	0.464
49	1 070	2 022	1 908	0.850	0.854	0.759	0.541
75	990	2 030	1 980	0.861	0.867	0.797	0.622
99	978	2 032	1 990	0.874	0.877	0.814	0.659

Tabella 4: Andamento della Fase 2 a intervalli di circa 25 step.

step 88). I Resistenti crescono con lo stesso andamento ma restano sempre leggermente sotto gli Infetti (1 990 contro 2 032 allo step 99).

Il grafico di destra mette a confronto due letture della segregazione, che in questa run crescono entrambe. Le metriche strutturali salgono in modo graduale e regolare: la modularità passa da 0.828 a 0.877 e l’Echo Chamber Index da 0.832 a 0.874. Le metriche di opinione crescono in modo molto più marcato: la belief assortativity sale da 0.27 a 0.66 (circa due volte e mezzo) e l’opinion homophily da 0.63 a 0.81, dopo un calo iniziale nei primi 4 step dovuto alla prima ondata di contagio. Il rewiring guidato dalla GNN rafforza quindi le community esistenti della rete di co-autoraggio e, in misura maggiore, raggruppa i nodi che condividono la stessa opinione.

Il rewiring sostituisce complessivamente 2 500 archi in 100 step, circa il 10% della rete, a densità costante (26 246 archi). Alla fine della Fase 2 la rete conta 86 componenti connesse invece di una: il vincolo di sicurezza impedisce di isolare singoli nodi, ma il rewiring può staccare piccoli gruppi dal resto della rete.

3.4 Fase 3: Intervento e Fact-Checking

L’ultima fase simula una contromisura strategica contro la diffusione della narrazione polarizzante.

- **Influence Maximization:** Il modulo d’intervento individua un set limitato di nodi (parametro CELF_BUDGET_K = 20) ideali per una campagna di contrasto, massimizzando il numero di agenti su cui la pressione dei fact-checker sarà sufficiente a produrre una transizione.
- **Azione sui vicini:** I *fact-checker* pubblicano contenuti correttivi nei feed dei vicini e, attraverso le soglie della macchina a stati, possono rendere resistenti sia i Suscettibili (inoculazione) sia gli Infetti.

La Fase 3 — *Deployment & Fact-Checking* — ha lo scopo di valutare l'efficacia delle strategie di contrasto alla disinformazione all'interno della rete sociale simulata. Introduce un sottoinsieme di agenti con il ruolo di Fact-Checker (stato F), con l'obiettivo di frenare l'infezione della narrazione polarizzante e spingere gli agenti infetti (I) o suscettibili (S) a diventare resistenti (R).

Disegno sperimentale: intervento e controllo. Un semplice confronto “prima/dopo” su una sola run mescolerebbe l'effetto dei fact-checker con l'evoluzione spontanea della rete, che continua a cambiare anche senza intervento. Per isolare l'effetto, la Fase 3 viene eseguita due volte a partire dallo stesso checkpoint di fine Fase 2 e per lo stesso numero di step: una run con l'iniezione dei fact-checker e una run di controllo senza (`CONTROL_RUN = True` nel notebook). Grazie al seed per richiesta dell'LLM, le due run ricevono le stesse risposte finché i prompt coincidono. L'effetto dell'intervento è la differenza tra le due run allo stesso step.

3.4.1 Il Flusso di Esecuzione della Fase 3

Il processo si sviluppa attraverso sette passaggi logici:

1. **Ripristino dello stato:** il notebook inizializza il `CheckpointManager` e l'orchestratore `SimulationOrchestrator` per ricaricare l'ultimo checkpoint salvato al termine della Fase 2 (co-evoluzione).
2. **Calcolo della baseline pre-intervento:** vengono calcolate le metriche strutturali e di opinione correnti (Infection Rate, modularità Q, Echo Chamber Index, polarizzazione, omofilia e assortatività delle opinioni) per registrare lo stato della rete prima dell'iniezione.
3. **Verifica della soglia e selezione dei seed:** l'intervento si attiva se l'infection rate supera `activation_threshold` (0.4); il parametro `FORCE_INJECTION` consente di iniettare comunque, registrandolo nel report. Vengono poi selezionati i k nodi in cui posizionare i fact-checker. Nella run di controllo questo passaggio e il successivo vengono saltati.
4. **Iniezione dei fact-checker:** la classe `FactCheckerInjector` converte lo stato dei nodi selezionati in F e vi inserisce un post iniziale di fact-checking per attivare la propagazione nei feed dei rispettivi vicini; lo stato degli agenti viene poi sincronizzato con il grafo (`sync_agent_states`).
5. **Simulazione post-intervento:** l'orchestratore fa avanzare la simulazione per N step post-intervento, ripartendo dallo step successivo all'ultimo della Fase 2. Durante questa fase gli agenti vicini ai nodi F leggono i post correttivi e, in base alle soglie e al loro livello cognitivo di suscettibilità, possono transitare in R. Gli step vengono eseguiti dalla stessa istanza dell'`SimulationOrchestrator`, con lo stesso LLM reale (server vLLM locale) impiegato nelle fasi precedenti.
6. **Confronto delle metriche:** viene calcolato il report finale con le metriche di fine run, le metriche dei fact-checker e le variazioni rispetto alla baseline pre-intervento.
7. **Salvataggio del report:** tutti i dati dell'intervento (inclusi i node ID scelti e la stima dei nodi coperti) vengono scritti in `results/phase3_report.json`, oppure in `results/phase3_report_control` per la run di controllo. L'ultima cella del notebook (*Confronto Intervento vs Controllo*) confronta infine le due run, usando solo i report con il flag `control_run` atteso e almeno uno step di Fase 3, e salva l'effetto dell'intervento in `results/phase3_comparison.json`.

3.4.2 Moduli Richiamati nella Fase 3

La Fase 3 si appoggia sul sottosistema di mitigazione dell'influenza ideologica (`src/influence/`) e riutilizza in modo coordinato i moduli precedentemente presentati.

Sottosistema src/influence/ — Algoritmi e Valutazione

- **celf.py** (selezione dei seed): esclude dal pool degli eleggibili i nodi già Infetti (I), già Fact-Checker (F) o già Resistenti (R) — iniettare un fact-checker su un nodo già resistente sarebbe uno spreco di budget. Supporta due obiettivi:
 - **threshold** (default): conta i nodi su cui la pressione dei fact-checker supera le stesse soglie che nella simulazione producono una transizione ($S \rightarrow R$ con almeno il 10% di vicini F, $I \rightarrow R$ con almeno il 25%). Il calcolo è deterministico ed esatto. Poiché una funzione a soglia non è submodulare, non si usa la valutazione lazy di CELF ma il greedy completo, il cui costo è comunque trascurabile.
 - **ic**: algoritmo CELF (*Cost-Effective Lazy Forward*) su modello Independent Cascade, con stime Monte Carlo (30 round) su numeri casuali comuni a tutti i candidati e guadagni marginali troncati a zero; le simulazioni sfruttano il parallelismo su CPU (`ProcessPoolExecutor`).
- **injector.py** (`FactCheckerInjector`): gestisce la logica di attivazione (`should_activate`, che fa scattare l'intervento solo se l'infection rate supera la soglia critica `activation_threshold`) e inietta lo stato F nei nodi seed selezionati. Forza la pubblicazione di un post di fact-checking iniziale nel `NetworkManager` (del tipo “[*FACT-CHECK seed*] *I have been designated to critically examine claims about '[Topic]'*...”), che funge da scintilla per la diffusione dell'informazione correttiva nei feed dei nodi adiacenti.
- **metrics.py** (metriche di influenza):
 - *Reach dei fact-checker* (`compute_fact_checker_spread`): numero di nodi entro `reach_hops` salti (default 1) da almeno un nodo F, cioè la platea che vede direttamente i contenuti correttivi.
 - *Copertura efficace* (`fc_effective_coverage`): frazione di nodi su cui la pressione dei fact-checker supera le soglie di transizione, cioè quelli che l'intervento può effettivamente convertire.
 - *Intervention Delta* (`compute_intervention_delta`): differenze (dopo – prima) delle metriche rispetto alla baseline pre-intervento.

Altri Sottosistemi Richiamati

- **orchestrator.py** (`SimulationOrchestrator`): coordina l'evoluzione del sistema negli step successivi all'iniezione dei fact-checker, aggiornando post, stati cognitivi e il rewiring della GNN; è la stessa istanza dell'orchestratore costruita per le Fasi 1 e 2 (parametro `USE MOCK_LLM = False` nel notebook), per cui gli agenti continuano a interrogare l'LLM reale anche durante il post-intervento.
- **network_manager.py** (`NetworkManager`): riceve i post di fact-checking dei seed e si assicura che vengano propagati nei feed degli agenti vicini.
- **metrics.py** (`compute_all_metrics`): calcola le metriche strutturali e di opinione all'inizio (baseline) e alla fine del post-intervento.
- **checkpoint.py** (`CheckpointManager`): fornisce i dati di ripristino per caricare la rete al termine della Fase 2 e avviare entrambi gli scenari della Fase 3.

Metrica (step 129)	Con fact-checker	Controllo	Effetto
Nodi S	798	974	-176 (di cui 20 seed divenuti F)
Nodi I	2020	2024	-4
Nodi R	2162	2002	+160
Infection Rate	0.404	0.405	-0.001
Belief Assortativity	0.704	0.692	+0.012
Opinion Homophily	0.831	0.831	0.000
Belief Polarisation	0.839	0.805	+0.034
Echo Chamber Index	0.886	0.886	0.000
Modularity Q	0.883	0.884	0.000
Transizioni in 30 step	177	17	+160

Tabella 5: Effetto dell'intervento allo step 129: run con 20 fact-checker a confronto con la run di controllo partita dallo stesso checkpoint.

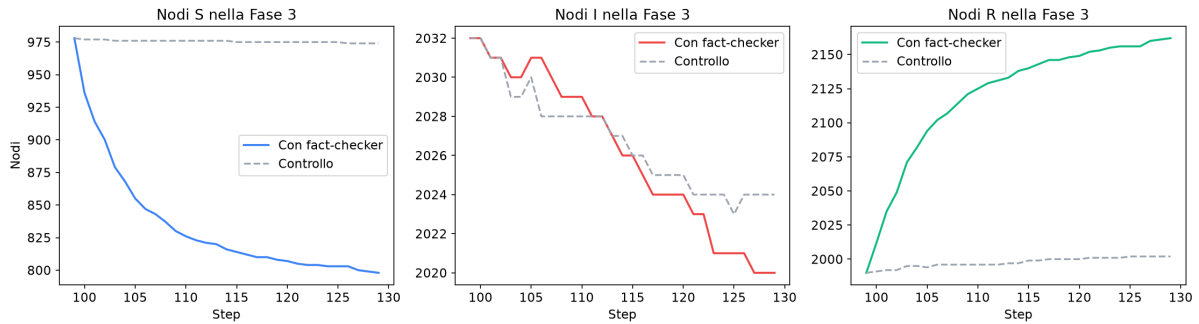


Figura 4: Evoluzione dei nodi Suscettibili (sinistra), Infetti (centro) e Resistenti (destra) dallo step 99 allo step 129, nella run con fact-checker (linea continua) e nella run di controllo (linea tratteggiata). Le scale verticali sono diverse: la variazione degli Infetti è di pochi nodi.

3.4.3 Risultati: Intervento a Confronto con il Controllo

Entrambe le run partono dal checkpoint dello step 99 ed eseguono 30 step post-intervento (step 100–129), con `CELF_BUDGET_K` = 20. L'intervento è partito con un infection rate di 0.406, sopra la soglia di attivazione (0.4), quindi senza forzature. La selezione con obiettivo `threshold` ha scelto 20 nodi suscettibili, con una stima di 162 nodi coperti.

Alla fine della run, i 20 fact-checker raggiungono direttamente 194 nodi (9.7 per fact-checker in media), mentre i nodi su cui la loro pressione supera ancora le soglie di transizione sono soltanto 5 (copertura efficace dello 0.10%): gran parte dei vicini raggiungibili è già stata convertita nei primi step.

3.4.4 Conclusioni sull'Efficacia dell'Intervento

Il confronto con il controllo restituisce un quadro netto su un fronte e più sfumato sugli altri.

- **Protezione dei suscettibili:** è l'effetto principale. I fact-checker convertono in resistenti 156 suscettibili in più rispetto al controllo (176 meno i 20 seed), quasi tutti nei primi 10 step: 135 delle 177 transizioni avvengono negli step 100–109, con 14–23 transizioni per step all'inizio e 0–4 alla fine. La stima della selezione (162 nodi coperti) coincide quasi esattamente con i resistenti in più rispetto al controllo (+160).
- **Infetti:** l'effetto è trascurabile, perché un infetto si converte solo se almeno un quarto dei suoi vicini è fact-checker. Gli infetti calano di poco in entrambe le run (da 2032 a 2020 con i fact-checker, a 2024 nel controllo): la differenza finale di -4 nodi si forma solo negli ultimi 15 step e non è distinguibile dalla variabilità della singola run.

Step	S (FC)	S (controllo)	I (FC)	I (controllo)
99	978	978	2 032	2 032
100	936	977	2 032	2 032
105	855	976	2 031	2 030
110	826	976	2 029	2 028
120	807	975	2 024	2 025
129	798	974	2 020	2 024

Tabella 6: Andamento di Suscettibili e Infetti nelle due run (FC = run con fact-checker).

- **Echo chamber:** l'intervento non le riduce e anzi aumenta leggermente la belief assortativity (+0.012). I nodi protetti diventano resistenti accanto ad altri resistenti, rafforzando i gruppi di opinione esistenti. Anche la polarizzazione cresce (+0.034), perché nodi neutrali (S) passano a una posizione netta (R).
- **Struttura:** Echo Chamber Index e modularità sono praticamente uguali nelle due run. L'intervento agisce sulle opinioni, non sulla topologia della rete.
- **Sproporzione tra budget e scala della rete:** 20 fact-checker rappresentano lo 0.4% della popolazione, introdotti quando gli infetti sono già oltre 2 000 e la loro crescita si è stabilizzata. Con le regole del modello, un budget così ridotto e tardivo può proteggere chi non è ancora stato contagiato, ma non recuperare una popolazione infetta già consolidata.

L'effetto sui suscettibili è netto. Quelli sugli infetti e sull'assortatività sono piccoli: poiché ogni configurazione è stata eseguita una sola volta, vanno letti con cautela e andrebbero confermati con più repliche.

4 Conclusioni e Prospettive

4.1 Visualizzazione della Rete: Topologia Iniziale vs Finale

A completamento dell'analisi quantitativa, è utile osservare direttamente l'evoluzione strutturale della rete attraverso un confronto visivo tra la topologia allo step 0 e quella al termine della simulazione (step 129), su un campione di 150 nodi centrati sugli hub principali del grafo.

Il confronto visivo conferma quanto osservato nelle metriche aggregate delle Fasi 2 e 3. Nella rete iniziale (sinistra) il campione si presenta come un insieme di piccoli cluster densi e ben separati tra loro, con pochissimi legami di lungo raggio: la struttura a compartimenti stagni tipica di una rete di co-autoraggio accademico organizzata per sotto-discipline. Nella rete finale (destra), a parità di nodi, emergono alcuni elementi distintivi:

- **Concentrazione degli archi sugli hub:** la rete nel suo complesso mantiene la stessa densità (26 246 archi), ma il campione di hub guadagna archi (da 1 345 a 1 588, +18%). Il rewiring sostituisce archi senza aggiungerne: quelli nuovi tendono a collegare nodi centrali, mentre quelli rimossi stanno altrove nella rete.
- **Cluster più compatti:** i nuovi archi si concentrano all'interno dei cluster già esistenti, che risultano più densi, invece di creare ponti tra gruppi diversi. È l'effetto del rewiring guidato dalla GNN: nuovi archi nascono tra agenti che hanno sviluppato posizioni simili nello spazio degli embedding, ed è coerente con la crescita di modularità ed Echo Chamber Index.
- **Stabilità della periferia:** i nodi periferici a basso grado (coppie isolate o singoli nodi) restano sostanzialmente invariati. Sui 150 hub osservati, il rewiring non stravolge la disposizione dei gruppi, ma ne aumenta la coesione interna.

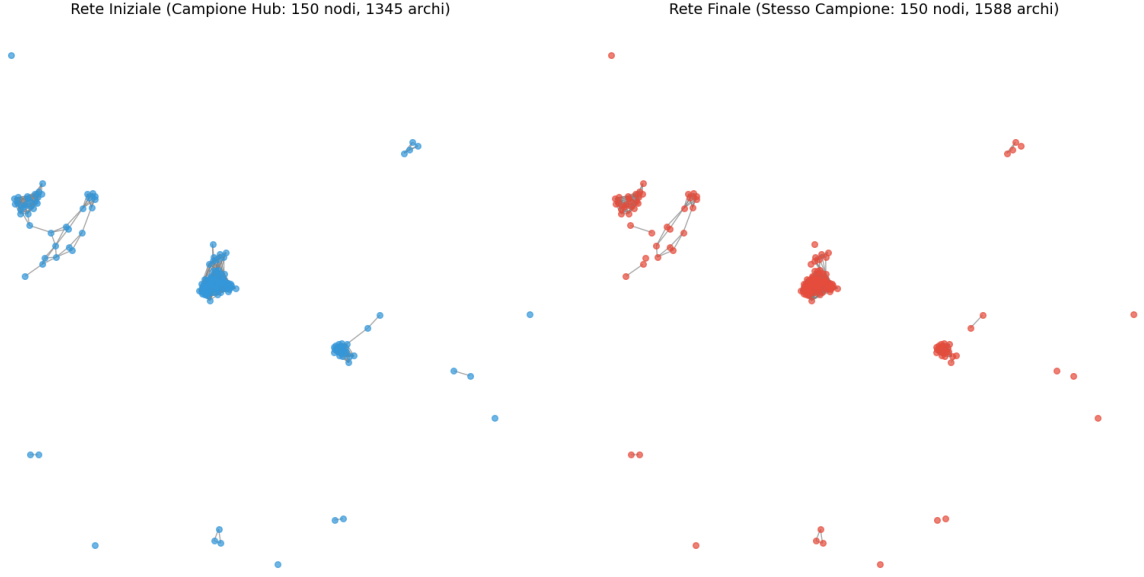


Figura 5: Confronto tra la topologia della rete allo step iniziale (sinistra, 150 nodi campionati sugli hub, 1345 archi) e allo step 129 (destra, stesso campione, 1588 archi), al termine della co-evoluzione. L’immagine si riferisce alla run di controllo; la topologia della run con intervento è praticamente identica, come confermano ECI e modularità.

Questo pattern visivo è coerente con il quadro emerso dalle metriche numeriche: il rewiring della GNN non stravolge la topologia della rete, ma la “rifinisce” selettivamente, sostituendo circa il 10% degli archi in 100 step e rendendo più coesi sia le community originarie sia, in misura maggiore, i gruppi di opinione. L’intervento di fact-checking, nella finestra osservata, agisce invece sul livello cognitivo (stati e opinioni) e non sulla struttura fisica del grafo.

4.2 Conclusioni Generali

La pipeline implementata in **progettoASM** rappresenta un solido paradigma di simulazione sociale in cui l’intelligenza artificiale generativa smette di essere un semplice strumento di supporto e diviene il motore stesso dell’evoluzione di rete.

L’architettura, attraverso la successione ordinata delle fasi, mostra come la co-evoluzione tra topologia e cognizione agentica non sia soltanto una curiosità teorica, ma un fenomeno misurabile e tracciabile quantitativamente. La coerenza strutturale del campionamento (verificata nella Fase 0), la scalabilità garantita dall’uso ottimizzato di vLLM (impiegato con continuità nelle Fasi 1, 2 e 3) e il disegno sperimentale con run di controllo conferiscono al framework una notevole robustezza metodologica.

I risultati principali si possono riassumere in quattro punti:

1. Gli agenti LLM producono una dinamica di diffusione realistica: crescita rapida, saturazione intorno al 41% dei nodi e comparsa spontanea di resistenti, che arrivano quasi a pareggiare gli infetti.
2. La co-evoluzione rafforza le echo chamber, soprattutto quelle di **opinione**: la belief assortativity sale da 0.27 a 0.66, mentre l’Echo Chamber Index strutturale cresce in misura molto minore (da 0.83 a 0.87). L’indice strutturale da solo sottostimerebbe la segregazione: è necessario affiancare metriche basate sulle opinioni.
3. Un intervento tardivo con 20 fact-checker (0.4% dei nodi) protegge circa 160 suscettibili (il 16% di quelli rimasti), in linea con la stima della selezione, ma lascia invariata la

popolazione infetta già consolidata. Con le regole del modello, il fact-checking funziona come prevenzione più che come cura.

4. L'intervento non riduce la polarizzazione: la rafforza leggermente, perché sposta nodi neutrali su posizioni nette.

Limitazioni. Ogni configurazione è stata eseguita una sola volta: con un costo di circa 6 minuti per step non è stato possibile ripetere le run con seed diversi, e gli effetti più piccoli richiederebbero repliche per essere confermati. Il seed per richiesta rende ripetibili le risposte a parità di prompt, ma vLLM non garantisce il determinismo completo con batching variabile. L'obiettivo di selezione dei seed considera l'effetto immediato dei fact-checker sui vicini, non la dinamica futura (ricadute, rewiring, risposte dell'LLM). Infine, il rewiring a densità costante ha staccato dalla rete 85 piccoli gruppi di nodi in 100 step (153 allo step 129): un vincolo di connettività sulle rimozioni renderebbe la rete più realistica.

I risultati ottenuti, archiviati in `metrics_history.csv`, `metrics_history_control.csv` e nei report della Fase 3, costituiscono la base per ulteriori indagini scientifiche. In futuro, il sistema potrà essere esteso integrando algoritmi di *GNN Explainer* per interpretare visivamente quali legami stiano effettivamente guidando la polarizzazione, testando diverse architetture di agenti LLM per osservare se differenti "personalità" algoritmiche conducano a velocità di convergenza verso l'omofilia sensibilmente diverse, o esplorando budget di fact-checker più ampi e finestre di attivazione più precoci, prima che il contagio si stabilizzi, per verificare a quali condizioni l'intervento riesca a ridurre anche la popolazione infetta.

4.3 Riepilogo Esecutivo della Pipeline

Al termine di ciascuna run, la pipeline stampa un riepilogo consuntivo che raccoglie in forma compatta i principali indicatori dell'esperimento, dalla costruzione del sottografo fino all'intervento finale. Di seguito i valori delle due run della Fase 3:

PIPELINE COMPLETATA		
	Con fact-checker	Controllo
Nodi simulati	5 000	5 000
Archi	26 246	26 246
Community	415 (Q baseline = 0.8265)	415 (Q baseline = 0.8265)
Step Fase 2	100	100
Infection rate	0.4064 → 0.4040	0.4064 → 0.4048
ECI	0.8741 → 0.8855	0.8741 → 0.8858
Seed fact-checker	20 nodi	0 nodi
Copertura efficace	0.0010	0.0000

Il riepilogo conferma, in forma aggregata, il quadro delineato nelle sezioni precedenti: la rete parte da una condizione di forte segregazione comunitaria (415 community, $Q = 0.8265$); nei 30 step finali l'infection rate scende di pochi millesimi in entrambe le run, mentre l'ECI cresce in modo quasi identico nelle due run, a riprova che l'intervento incide sulla dimensione cognitiva e non su quella strutturale della rete.